Methodology

A. Data and Target Variable

The input data is the table cleaned_customer_data in MySQL (database dddm_telco_churn), which contains 7,043 customers, one row per customer. The target variable is **churn_flag**: 1 if the customer churned and 0 if the customer stayed. The churn rate of the full dataset is 26.54% (1,869 customers) and the retention rate is 73.46%. This is a binary classification problem on imbalanced data: churners are the minority class, but they are the group the business cares about most.

B. Input Features (18 Variables)

Table 1. The 18 input variables of the model

Group	Variables	Count
Demographics	gender, SeniorCitizen, Partner, Dependents	4
Phone and internet services	PhoneService, MultipleLines, InternetService, OnlineSecurity, OnlineBackup, DeviceProtection, TechSupport, StreamingTV, StreamingMovies	9
Contract and payment	Contract, PaperlessBilling, PaymentMethod	3
Tenure and charges	tenure, MonthlyCharges	2

tenure (months of service) and MonthlyCharges (monthly fee) are numeric variables. All other variables are categorical, and their column types are converted in KNIME before training. For a categorical variable with several levels, the model compares each level with a reference level. For example, Contract = One year and Contract = Two year are both compared with the month-to-month contract.

C. Columns Excluded from the Input and Why

Table 2. The five columns excluded from the model

Column	Reason for exclusion
customerID	An identifier with no predictive information. Keeping it would let the model memorize individual customers.
Churn	The target variable in text form (Yes/No). Keeping it would put the answer into the input and cause data leakage.
churn_flag	The target variable in 0/1 form. Same reason as Churn.
tenure_group	A grouped version of tenure (four non-overlapping bins: 0-12, 13-24, 25-48 and 49-72 months), so it duplicates information. Keeping both increases multicollinearity without adding information.
TotalCharges	Approximately equal to tenure multiplied by MonthlyCharges: for 94.5% of customers with a positive TotalCharges, the two values differ by at most 10%. It therefore depends almost linearly on two variables

	already in the model. Excluded for now so that the coefficients of tenure and MonthlyCharges stay interpretable.

The column tenure_group stays in the file churn_dashboard_data.csv to build charts on the dashboard, but it does not enter the model. These exclusions reduce redundancy among the inputs, but they do not remove all multicollinearity: Section I of Experiments and Results shows that some of the remaining variables (InternetService, MonthlyCharges) still show signs of it.

D. Experimental Design: Train and Test Split
The data is split 80/20, stratified by churn_flag, with seed 42.

Table 3. Result of the data split

Set	Customers	Actual churners	Churn rate
Training (train)	5,634	1,495	26.54%
Test	1,409	374	26.54%

The number of actual churners in each set comes from the query result on the table churn_predictions in MySQL (Figure 5).

Stratification means that each set keeps the same churn rate as the full data. This matters because churners are only 26.54% of customers: with a purely random split, the test set could contain too few or too many churners, which would make recall and precision fluctuate. Seed 42 fixes the split, so anyone who reruns the workflow obtains the same two sets.

E. Preventing Data Leakage
Data leakage happens when the model has access to information that would not be available at prediction time, which makes the evaluation look better than reality. We apply the following measures. Measures 1, 4 and 5 prevent data leakage directly. Measures 2 and 3 are not leakage controls; they stop the model from memorizing customers and remove duplicated information:

1. Exclude the target variable and every copy of it: Churn and churn_flag do not enter the input.
2. Exclude customerID so the model cannot memorize individual customers (this is not data leakage).
3. Exclude columns derived from other variables (tenure_group, TotalCharges) to avoid duplicated information and multicollinearity (this is not data leakage).
4. Split into train and test before training. The model learns its coefficients on the train set only, and the test set does not take part in training.
5. Measure every evaluation metric (confusion matrix, recall, precision, accuracy, AUC) on the test set, not on the train set.

One point must be stated plainly: the threshold 0.25 was chosen using the results on the test set itself (see Limitations), so this is a mild form of leakage in the threshold selection step. Measures 1, 4 and 5 remove leakage in training and evaluation, but they cannot remove this one.

F. Model: Logistic Regression

We use Logistic Regression with the IRLS solver (iteratively reweighted least squares), trained in KNIME. The model estimates the probability that a customer churns, where the log-odds are a linear combination of the input variables. We chose it for three reasons: (1) its output is a probability between 0 and 1, which suits ranking customers by risk and setting an alert threshold; (2) each coefficient can be read as an odds ratio (the exponential of the coefficient), which fits the decision questions DQ1 and DQ2 that ask which factors influence churn; (3) the model is simple and easy to present to non-specialists.

Figure 1 summarizes the KNIME workflow Week3_ML_Churn. The workflow reads the table cleaned_customer_data from MySQL (MySQL Connector, DB Table Selector, DB Reader), converts column types (Number to String), keeps the input variables (Column Filter), and performs the stratified 80/20 split with seed 42 (Table Partitioner). The Logistic Regression Learner is trained on the train set only. Two Logistic Regression Predictor nodes compute churn probabilities for the train set and the test set. On the test branch, two Scorer nodes produce the confusion matrix and the metrics: one reads the prediction at the default threshold 0.50, the other follows a Rule Engine that applies threshold 0.25. A ROC Curve node computes the AUC. The outputs of both sets are concatenated, and Rule Engine, Column Renamer, Column Filter and String to Number nodes prepare the output columns, including the dataset_split label and the predicted_churn label (threshold 0.25). A DB Writer node then writes the table churn_predictions to MySQL through a second MySQL Connector, together with a DB SQL Executor node.

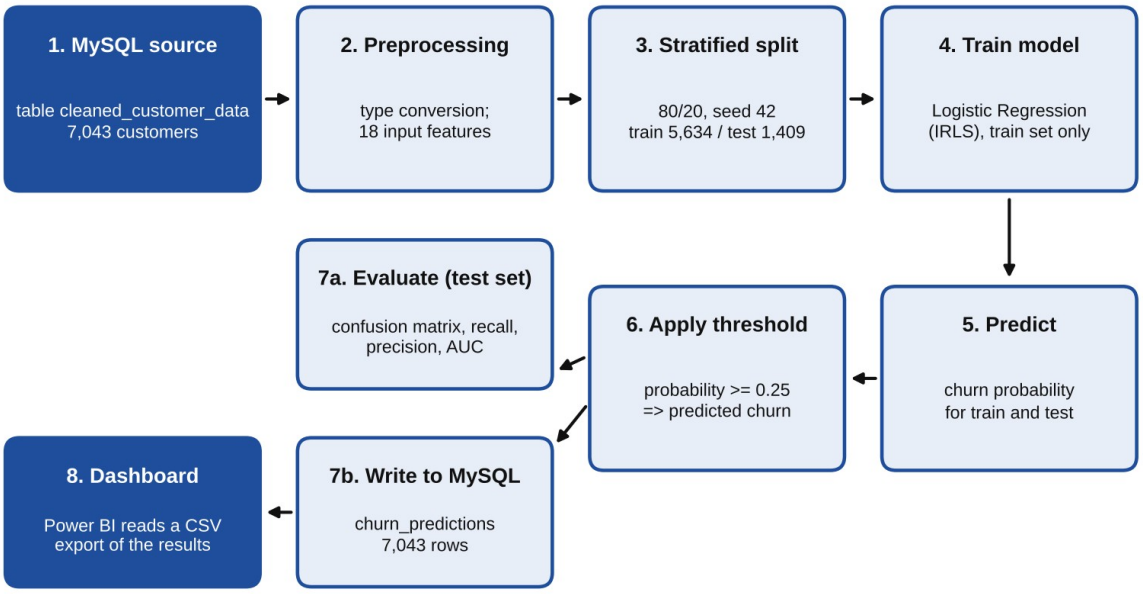


Figure 1. ML workflow from the MySQL source table to the dashboard (KNIME workflow Week3_ML_Churn)

G. Handling Class Imbalance with the Classification Threshold

The data is imbalanced (73.46% stayed, 26.54% churned). Instead of rebalancing the data, we lower the classification threshold from 0.5 to 0.25 to increase recall. The business reason: missing a customer who is about to leave (losing the customer) costs more than wrongly flagging a customer who would have stayed (an unnecessary retention cost). We compare three thresholds: 0.50, 0.30 and 0.25.

H. Evaluation Metrics and the Targets Set in Week 1

Here TP is the number of churners the model catches, FN is the number of churners it misses, and FP is the number of customers who stayed but were flagged by mistake.

- **Recall** = TP / (TP + FN): of the customers who actually churned, the percentage the model catches.
- **Precision** = TP / (TP + FP): of the customers the model flags, the percentage who actually churn.
- **Accuracy**: the share of correct predictions over the whole test set.
- **AUC-ROC**: the probability that the model gives a higher score to an actual churner than to a customer who stayed, when one customer of each kind is picked at random. AUC does not depend on the threshold.

Targets from Week 1: **recall of at least 0.80** and **AUC-ROC of at least 0.75**. Recall is the main metric for the reason given in Section G of this Methodology. Accuracy is not used as the main metric: a model that predicts that every customer stays still reaches an accuracy of 73.46% without catching a single churner.

Experiments and Results

All numbers below are measured on the test set: 1,409 customers, of which 374 actually churned and 1,035 stayed.

A. Comparison of Three Thresholds

Table 4. Results of three classification thresholds on the test set

Threshold	TP	FN	FP	Recall	Precision	Accuracy
0.50	210	164	99	0.561	0.680	0.813
0.30	291	83	232	0.778	0.556	0.776
0.25 (chosen)	**306**	**68**	**292**	**0.818**	**0.512**	**0.744**

The row for 0.25 is verified by screenshots: the confusion matrix has 743 customers who stayed and were predicted correctly, 292 false alarms, 68 missed churners and 306 correctly caught churners (Figure 2); the metrics table gives recall 0.818, precision 0.512 and accuracy 0.744 (Figure 3). Figure 3 also shows specificity 0.718, F-measure 0.63 and Cohen's kappa 0.45.



Figure 2. Confusion matrix at threshold 0.25

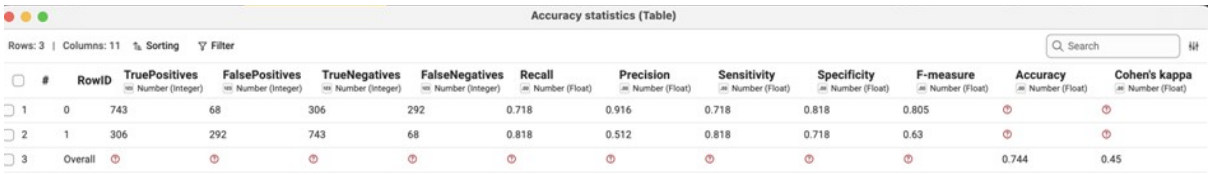


Figure 3. Evaluation metrics at threshold 0.25

Note: accuracy at threshold 0.25 is 0.744 (1,049 correct predictions out of 1,409), as shown in Figure 3.

B. Why the Final Recall Differs from the Value Before Threshold Adjustment

Recall changes with the threshold; it is not a fixed number.

At the default threshold of 0.50, the model reaches a recall of only 0.561: it catches 210 of 374 churners and misses 164. This is 0.239 below the 0.80 target, so keeping the default threshold would not meet the Week 1 target. The cause is not the model's ability to separate the two groups, since the AUC-ROC already reaches 0.860. The cause is the class imbalance: only

26.54% of customers churn, so the probability the model assigns to churners often stays below 0.5, even though those customers still rank higher than customers who stayed. In other words, the model ranks well, but the cut-off at 0.5 is poorly placed.

When the threshold drops to 0.30, recall rises to 0.778 but still misses the 0.80 target. Only at 0.25 does recall reach 0.818 (306 caught, 68 missed) and exceed the target. Among the three thresholds tried, 0.25 is the highest one that meets the recall target, so it keeps the best precision among the thresholds that meet the target.

Therefore, the different recall values across stages come mainly from different thresholds, not from a change in the model: 0.561 at threshold 0.50, 0.778 at 0.30 and 0.818 at 0.25. The official result is recall 0.818 at threshold 0.25.

C. Trade-offs of Lowering the Threshold

- Recall rises from 0.561 to 0.818, so the model catches **96** more churners (306 minus 210). The number of missed churners falls from 164 to 68.
- The cost is that precision falls from 0.680 to 0.512 and false alarms rise from 99 to 292 (193 more customers).
- Accuracy falls from 0.813 to 0.744. This is an intended consequence of prioritizing recall, not a sign that the model got worse.
- The total number of flagged customers rises from 309 to 598, that is, from about 22% to about 42% of the test set. With a precision of 0.512, about one in two flagged customers actually churns, which is about 1.9 times the overall churn rate of 26.54%. (The figures 309, 598, 22%, 42% and 1.9 times are computed from the numbers in Table 4.)

Business meaning: the company catches 306 of 374 customers who are about to leave, but it must accept spending retention effort on many customers who did not intend to leave. This is reasonable when the cost of retaining a customer is lower than the cost of losing one. If the retention budget is limited, the company can prioritize the customers with the highest churn_probability in the flagged list (the priority customer table on the dashboard).

D. AUC-ROC

The AUC-ROC of the model is 0.860, above the 0.75 target (Figure 4). Interpretation: if we pick one actual churner and one customer who stayed at random, the model gives the churner a higher probability in 86% of the cases. The blue ROC curve lies far above the dashed diagonal (random guessing, AUC 0.5), which shows that the model separates the two groups well. The operating point at threshold 0.25 corresponds to a true positive rate of 0.818 and a false positive rate of about 0.282 (292 of 1,035 customers who stayed, computed from the numbers in Table 4).

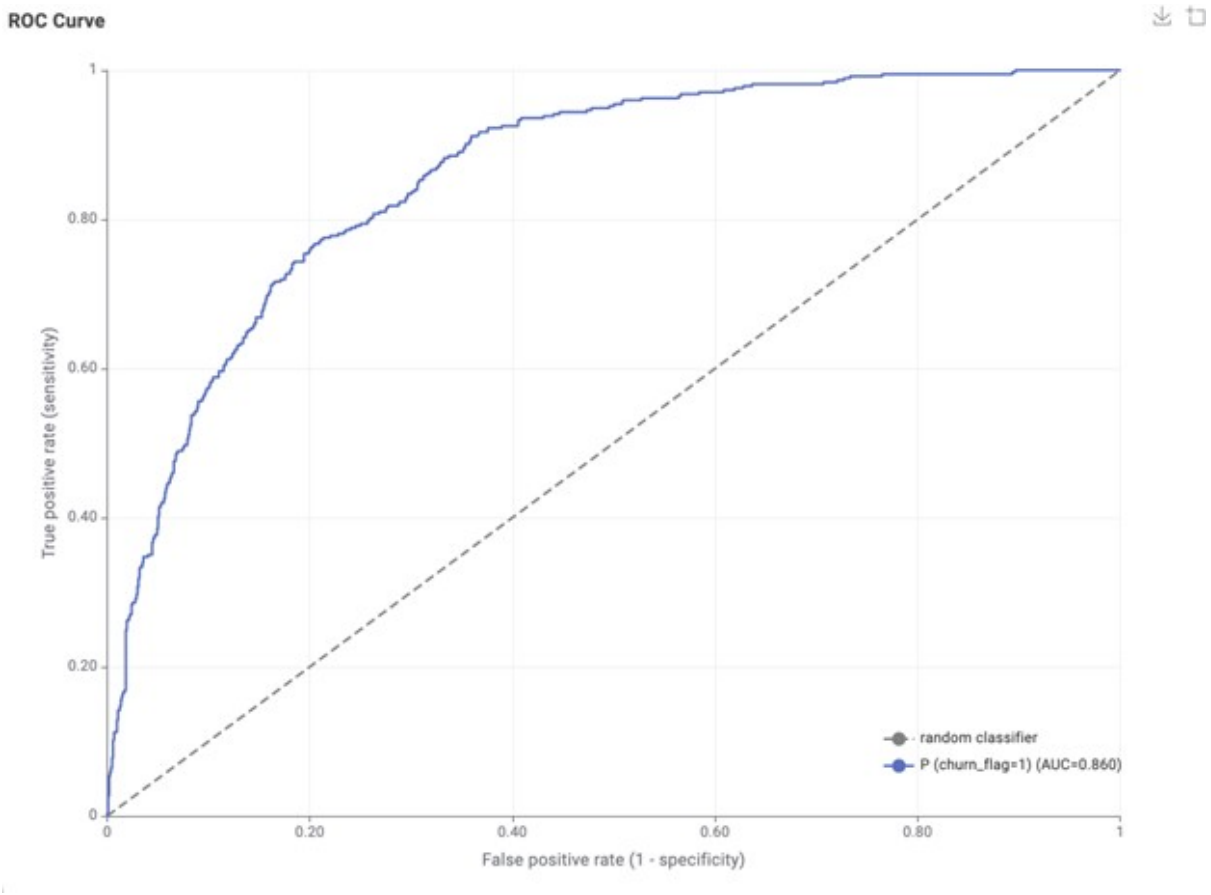


Figure 4. ROC curve and AUC = 0.860

Because AUC does not depend on the threshold, it shows that the model ranks customers by risk well whether we choose 0.25 or 0.30. It is also the basis for the conclusion in Section B: the problem at threshold 0.5 is the cut-off, not the ranking quality. We should add that this AUC is measured on a single test set, so it is supporting evidence, not absolute proof.

E. Comparison with the Week 1 Targets

Table 5. Results compared with the targets

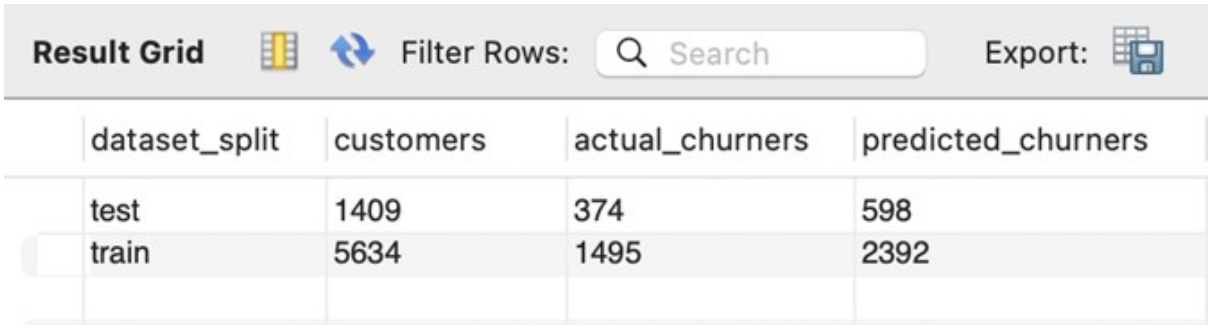
Metric	Week 1 target	Result	Met?
Recall (threshold 0.25)	at least 0.80	0.818	Met
AUC-ROC	at least 0.75	0.860	Met

Both targets are met. Recall meets the target only at threshold 0.25; at 0.50 (0.561) and 0.30 (0.778) it does not.

F. Storing the Results in MySQL

The predictions are written to the table churn_predictions with 7,043 rows, which matches the number of customers in the source data. The train set has 5,634 customers (1,495 actual churners, 2,392 predicted as churners) and the test set has 1,409 customers (374 actual churners, 598 predicted as churners) (Figure 5). The share of flagged customers is almost the

same in both sets (about 42.5% in train and 42.4% in test). This table is the source for the file churn_dashboard_data.csv used by the dashboard.

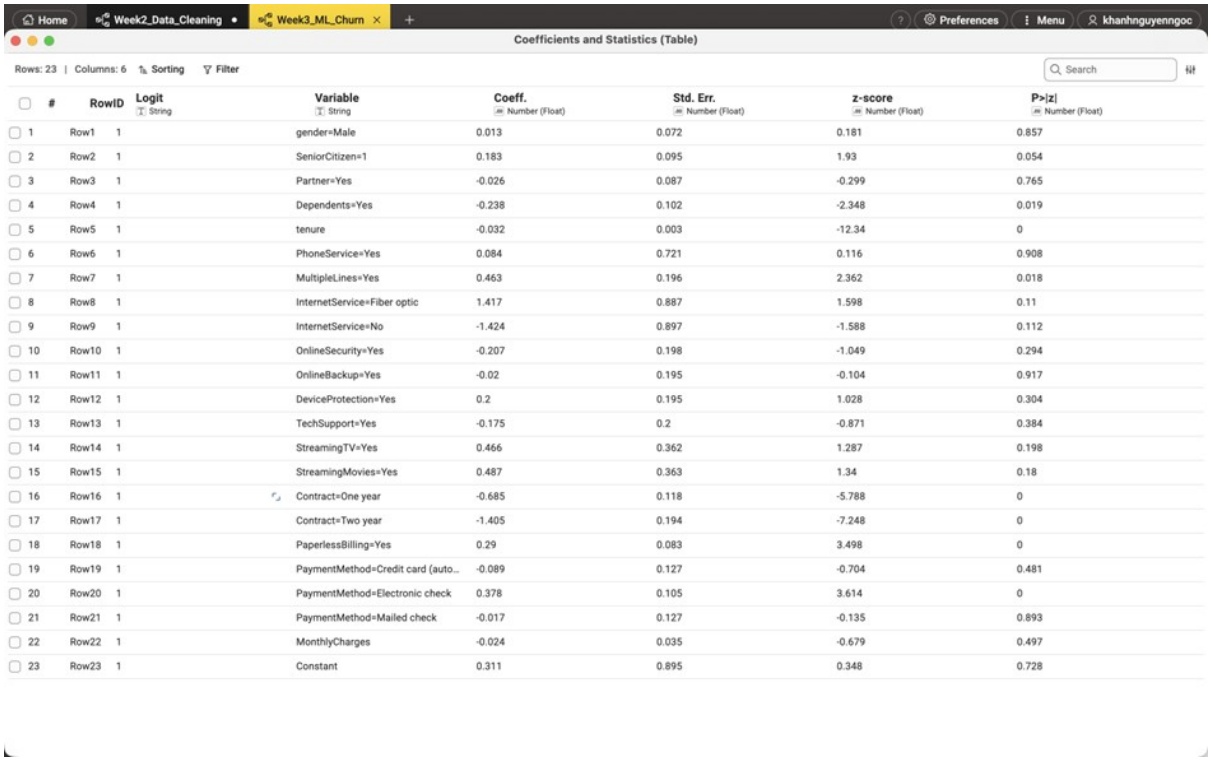


dataset_split	customers	actual_churners	predicted_churners
test	1409	374	598
train	5634	1495	2392

Figure 5. Query results of the table churn_predictions grouped by dataset_split

G. Interpretation of Model Coefficients

A positive coefficient increases the likelihood of churn and a negative coefficient decreases it. The odds ratio equals the exponential of the coefficient: a value above 1 increases the odds of churn and a value below 1 decreases them. Figure 6 shows the full coefficient table of the model.



#	RowID	Logit	Variable	Coeff.	Std. Err.	z-score	P>|z|
1	Row1	1	gender=Male	0.013	0.072	0.181	0.857
2	Row2	1	SeniorCitizen=1	0.183	0.095	1.93	0.054
3	Row3	1	Partner=Yes	-0.026	0.087	-0.299	0.765
4	Row4	1	Dependents=Yes	-0.238	0.102	-2.348	0.019
5	Row5	1	tenure	-0.032	0.003	-12.34	0
6	Row6	1	PhoneService=Yes	0.084	0.721	0.116	0.908
7	Row7	1	MultipleLines=Yes	0.463	0.196	2.362	0.018
8	Row8	1	InternetService=Fiber optic	1.417	0.887	1.598	0.11
9	Row9	1	InternetService=No	-1.424	0.897	-1.588	0.112
10	Row10	1	OnlineSecurity=Yes	-0.207	0.198	-1.049	0.294
11	Row11	1	OnlineBackup=Yes	-0.02	0.195	-0.104	0.917
12	Row12	1	DeviceProtection=Yes	0.2	0.195	1.028	0.304
13	Row13	1	TechSupport=Yes	-0.175	0.2	-0.871	0.384
14	Row14	1	StreamingTV=Yes	0.466	0.362	1.287	0.198
15	Row15	1	StreamingMovies=Yes	0.487	0.363	1.34	0.18
16	Row16	1	Contract=One year	-0.685	0.118	-5.788	0
17	Row17	1	Contract=Two year	-1.405	0.194	-7.248	0
18	Row18	1	PaperlessBilling=Yes	0.29	0.083	3.498	0
19	Row19	1	PaymentMethod=Credit card (auto...	-0.089	0.127	-0.704	0.481
20	Row20	1	PaymentMethod=Electronic check	0.378	0.105	3.614	0
21	Row21	1	PaymentMethod=Mailed check	-0.017	0.127	-0.135	0.893
22	Row22	1	MonthlyCharges	-0.024	0.035	-0.679	0.497
23	Row23	1	Constant	0.311	0.895	0.348	0.728

Figure 6. Coefficients and statistics of the model

Variables that are statistically significant (P below 0.05), taken from Figure 6:

Table 6. Statistically significant variables

Variable	Coefficient	Odds ratio	P
tenure (per month)	-0.032	0.97	below 0.001
Contract = One year	-0.685	0.50	below 0.001

Contract = Two year	-1.405	0.25	below 0.001
PaymentMethod = Electronic check	0.378	1.46	below 0.001
PaperlessBilling = Yes	0.290	1.34	below 0.001
MultipleLines = Yes	0.463	1.59	0.018
Dependents = Yes	-0.238	0.79	0.019

Interpretation by factor, holding the other variables constant:

- **Tenure:** each additional month of service lowers the odds of churn by about 3% (odds ratio 0.97). Cumulatively, a customer who stays 12 more months has odds of churn of about 0.68 times (computed as e to the power of -0.032 times 12). This agrees with Week 2: the 0 to 12 month group churns at 47.44%, while the 49 to 72 month group churns at only 9.51%.
- **Contract type:** compared with a month-to-month contract, a one-year contract has odds of churn of 0.50 times and a two-year contract 0.25 times. This agrees with Week 2 (month-to-month 42.71%, one year 11.27%, two years 2.83%). It is the factor with the largest coefficient in the model.
- **Payment and billing:** paying by electronic check raises the odds of churn by 1.46 times, and paperless billing raises them by 1.34 times. Electronic check agrees with Week 2 (45.29% churn, compared with 15.24% for credit card).
- **Multiple lines (MultipleLines = Yes):** the odds of churn rise by 1.59 times, with weaker significance (P = 0.018).
- **Dependents (Dependents = Yes):** the odds of churn fall to 0.79 times (P = 0.019), which suggests that customers with a family stay longer.

Note: the coefficients show a statistical association, not a causal relationship. The model does not tell us whether moving customers to a longer contract would surely make them stay.

H. Answers to DQ1 and DQ2

DQ1 (customer segments with the highest churn risk): the highest-risk segment is customers on a month-to-month contract with short tenure. One-year and two-year contracts reduce the odds of churn to 0.50 and 0.25 times those of the month-to-month contract, and each additional month of tenure reduces the odds by about 3%. Customers who pay by electronic check and use paperless billing also have a higher risk. Monthly charges (MonthlyCharges) are not statistically significant in the model (coefficient -0.024, P = 0.497), so we cannot say that high charges drive customers away once contract and tenure are taken into account.

DQ2 (add-on services that reduce churn risk): the model does not provide enough evidence to conclude. Both coefficients are negative (OnlineSecurity = Yes: -0.207, odds ratio about 0.81, P = 0.294; TechSupport = Yes: -0.175, odds ratio about 0.84, P = 0.384) but neither is statistically significant. Therefore the report states only the direction of the

coefficients and does not claim that these two services retain customers. The reason is explained in Section I.

I. Variables That Are Not Statistically Significant

Fiber optic (coefficient 1.417, P = 0.110), OnlineSecurity, TechSupport and MonthlyCharges are not statistically significant in this model, even though the Week 2 groups differ widely. The reason: the Week 2 figures compare one variable at a time, while the coefficients here measure the effect of each variable while holding the others constant. Part of the effect of these variables may already be explained by the other variables (especially contract type and tenure).

The coefficient table also shows signs of multicollinearity: very large standard errors for InternetService = Fiber optic (0.887), InternetService = No (0.897), PhoneService (0.721) and the constant (0.895), while the coefficients of the two InternetService levels are almost exact opposites (1.417 and -1.424). Monthly charges are closely related to the internet service type and the bundled services, so these variables may mask one another. We have not tested the VIF, so this is only a sign, not a conclusion. In addition, SeniorCitizen has P = 0.054, close to the 0.05 level but not below it, so we do not count it as significant.

Limitations

1. The threshold 0.25 was chosen using the results on the test set itself, so the recall of 0.818 may be slightly optimistic compared with completely new data. Improvement: choose the threshold on a separate validation set.
2. The model uses a single data split (seed 42) and has no cross-validation, so we do not know how stable the results are across different splits.
3. Multicollinearity has not been tested with the VIF, so the interpretation of the coefficients of non-significant variables needs caution.
4. We tried only three thresholds (0.50, 0.30, 0.25) and did not scan the full range of thresholds.
5. A precision of 0.512 means that almost half of the alerts are false alarms, so the cost of a retention campaign must be weighed in real use.